LeaseI

Agentic AI Support Investigation Platform

Engineering Case Study

Author	Venkata Lakshmi Parimala Pasupuleti
Domain	Affordable Housing / Enterprise Support Engineering
Status	Current build with explicitly separated target architecture
Primary Technologies	Python, FastAPI, LangGraph, SQLite, TF-IDF, HTTP tool clients
Core Safety Boundary	Read-only diagnosis; no automatic eligibility or configuration decisions

Document Integrity Standard

Every capability in this case study is explicitly identified as BUILT, PARTIALLY BUILT, or TARGET. The distinction is part of the architecture, not an editorial footnote.

Contents

1. Executive Summary
2. Origin and Business Problem
3. Product Scope and Users
4. Architecture Principles
5. Four-Source Knowledge and Tool Model
6. Current System Architecture
7. FastAPI Application Layer
8. LangGraph Orchestration and State
9. Triage and Conditional Routing
10. Configuration Agent
11. Historical Incident Agent
12. Structured Data and HUD Income-Limit Tool
13. Retrieval Architecture
14. Reasoning and Root-Cause Ranking
15. Validation and Evidence Grounding
16. Confidence and Escalation
17. Tenant Safety Boundary
18. Data Model and Storage
19. API and Execution Model
20. Worked Investigation
21. Testing Strategy
22. Reliability and Failure Handling
23. Security, Privacy, and Access Control
24. Observability and Evaluation
25. Deployment and Scaling
26. Design Decisions and Trade-offs
27. Current Limitations
28. Modernization Roadmap
29. Lessons Learned
Appendix A. Status Matrix
Appendix B. Shared State Reference
Appendix C. Glossary

1. Executive Summary

LeaseI is an agentic AI platform designed to support Affordable Housing document analysis and production issue investigation. Its purpose is not to replace the existing compliance engine or allow a language model to decide whether a household is eligible. Its purpose is to gather evidence from multiple systems, connect product behavior with regulatory and historical context, rank likely root causes, and hand a support engineer or developer a defensible investigation result.

The central architectural decision is to avoid treating all enterprise knowledge as one generic vector database. LeaseI separates four sources because each has a different freshness, trust, and access model: product and engineering knowledge; federal HUD and LIHTC regulatory knowledge; historical incidents; and live structured-data tools. Unstructured knowledge is retrieved. Current values are fetched through tools. Deterministic application logic remains authoritative.

The current implementation is intentionally small but structurally meaningful. FastAPI exposes the application endpoints. LangGraph coordinates a stateful workflow. Configuration, historical incident, structured data, and income-limit lookup nodes collect evidence. A reasoning node ranks causes, a validation node removes unsupported claims, and a threshold-based escalation path creates a structured packet when evidence is weak.

The current build uses SQLite, synchronous execution, keyword-based triage, TF-IDF similarity, deterministic graph routing, and hand-tuned confidence constants. These choices are appropriate for the present project scale, but they are not presented as production-complete enterprise infrastructure. The target design introduces structured ticket extraction, hybrid retrieval, formal validated tool requests, PostgreSQL and pgvector, asynchronous workers, role-based access controls, and evaluation-driven confidence signals.

BUILT - FastAPI, LangGraph state, specialized nodes, validation, escalation, tenant safety checks, SQLite persistence, and the core worked investigation are implemented and tested.

TARGET - Enterprise-grade retrieval, identity and access, asynchronous execution, real regulatory corpus indexing, live production integrations, and measured confidence are designed but not yet implemented.

2. Origin and Business Problem

The platform was motivated by a real production investigation. A customer appeared ineligible for Affordable Housing. The expected debugging path was to validate income limits, household composition, program rules, effective dates, certification inputs, and the eligibility calculation. Those checks were performed and the Affordable Housing module appeared correct.

The actual cause was outside the module: a system-level property setting introduced or changed by another team. The setting needed to be disabled for the affected Affordable Housing properties, but the dependency was not visible to the team investigating the problem. Nearly a full day was spent examining correct business logic because the relevant organizational knowledge was fragmented.

This incident exposed a recurring enterprise problem. Product behavior is distributed across code, property settings, release history, team knowledge, historical tickets, regulatory rules, and live values. A support ticket rarely arrives with enough context to know which of those sources matters. The investigation cost comes from discovering the dependency graph, not merely reading the rule.

Problem Statement

Reduce the time required to diagnose complex Affordable Housing support issues by combining product knowledge, regulatory context, historical incidents, and current structured values into one auditable investigation workflow.

3. Product Scope and Users

LeaseI serves two connected workflows: document intelligence and support investigation. Both use the same evidence model, but they have different risk profiles and output requirements.

Workflow	Primary Users	Outcome
Document intelligence	Support, QA, implementation, product	Grounded answers, clause or rule retrieval, cited explanations
Support investigation	Support engineers, developers, QA, product operations	Ranked root causes, checks performed, live values, related incidents, escalation packet
Tenant self-service	Residents or tenant-facing users	Safe information and escalation; no automatic regulated determination

3.1 In Scope

- Ticket classification and routing
- Configuration and release-history lookup
- Historical incident retrieval
- Current income-limit lookup
- Evidence-based root-cause ranking
- Unsupported-claim validation
- Structured human escalation
- Read-only tenant-facing safety controls

3.2 Explicitly Out of Scope

- Changing production settings
- Writing directly to customer or certification records
- Replacing the deterministic compliance engine
- Automatically approving or denying eligibility
- Presenting model-generated legal or regulatory interpretations as authoritative

4. Architecture Principles

Separate retrieval from tools

Documents and incidents may be retrieved from an index. Current income limits and settings must be fetched from authoritative structured sources.

Deterministic source of truth

The AI layer explains and investigates. Existing rules and human staff make regulated determinations.

Evidence before confidence

A claim is retained only when the workflow can point to supporting evidence.

Read-only by design

Investigation tools have no write path into production settings or regulated data.

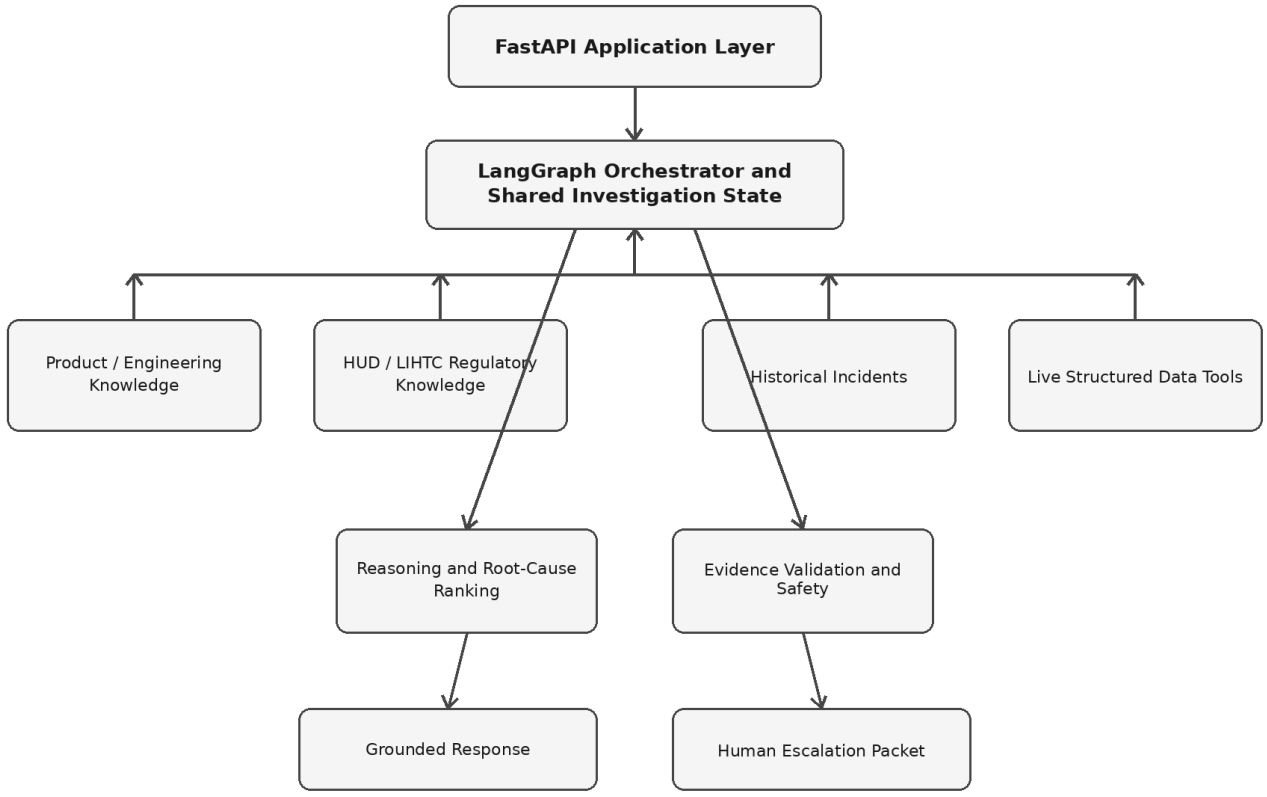
Progressive complexity

SQLite and synchronous execution are acceptable until scale or product requirements justify distributed infrastructure.

Human escalation is a valid result

The workflow is designed to say that evidence is insufficient instead of forcing a confident answer.

5. Four-Source Knowledge and Tool Model



Source	Current Access Pattern	Trust / Freshness	Status
Product and engineering knowledge	Structured SQLite queries for property settings and release notes	Current within local project data; broader prose documentation not indexed	Partially built

Source	Current Access Pattern	Trust / Freshness	Status
HUD and LIHTC regulatory knowledge	Hand-coded compliance rules and synthetic summaries	Deterministic rules are usable; authoritative corpus retrieval is absent	Target corpus
Historical incidents	Keyword overlap over SQLite tickets	Useful for repeated vocabulary; weak for paraphrases and unresolved tickets	Partially built
Live structured data	HUD User API client plus local SQLite lookups	Current at request time; external production systems not yet integrated	One real tool built

5.1 Product and Engineering Knowledge

The Configuration Agent queries structured property settings and release notes directly. This is intentionally not embedded because the data already has stable fields such as property, setting name, value, release, module, and effective context. Semantic retrieval is reserved for future technical prose such as architecture guides, API documentation, and support runbooks.

5.2 Regulatory Knowledge

Current regulatory behavior is represented by deterministic rule logic inherited from the original PropLens build and synthetic rule summaries in seed data. The production target is a separately indexed corpus of real HUD and LIHTC text with metadata for program, rule type, section, jurisdiction, effective year, and publication version.

5.3 Historical Incidents

Historical tickets provide organizational memory. Raw tickets are not equally trustworthy: some may be unresolved, speculative, or incomplete. The schema therefore includes a knowledge_entries concept for confirmed, staff-validated root causes. The feedback loop that promotes a resolved incident into this higher-trust tier is designed but not yet active.

5.4 Live Structured Data

The HUD income-limit client is a real HTTP integration using bearer-token authentication and the documented endpoint structure. Property settings, feature flags, and certification-like values are current within SQLite but do not yet represent production-system integrations. This distinction is important: functionally current local data is not the same as enterprise integration.

6. Current System Architecture

BUILT - Application endpoints, LangGraph state management, deterministic routing, specialized data-access nodes, reasoning, validation, escalation, and safety tests exist in the current codebase.

The current application is a modular monolith. FastAPI owns the HTTP boundary. Python functions own all database and external-service access. LangGraph defines the investigation sequence. SQLite stores

operational and knowledge records. The LLM receives only evidence already collected by application-owned functions.

This architecture avoids premature service distribution. It still enforces meaningful boundaries between API handling, orchestration, retrieval, tool access, reasoning, validation, and escalation. Those boundaries make later extraction into independently deployed components possible without requiring that complexity today.

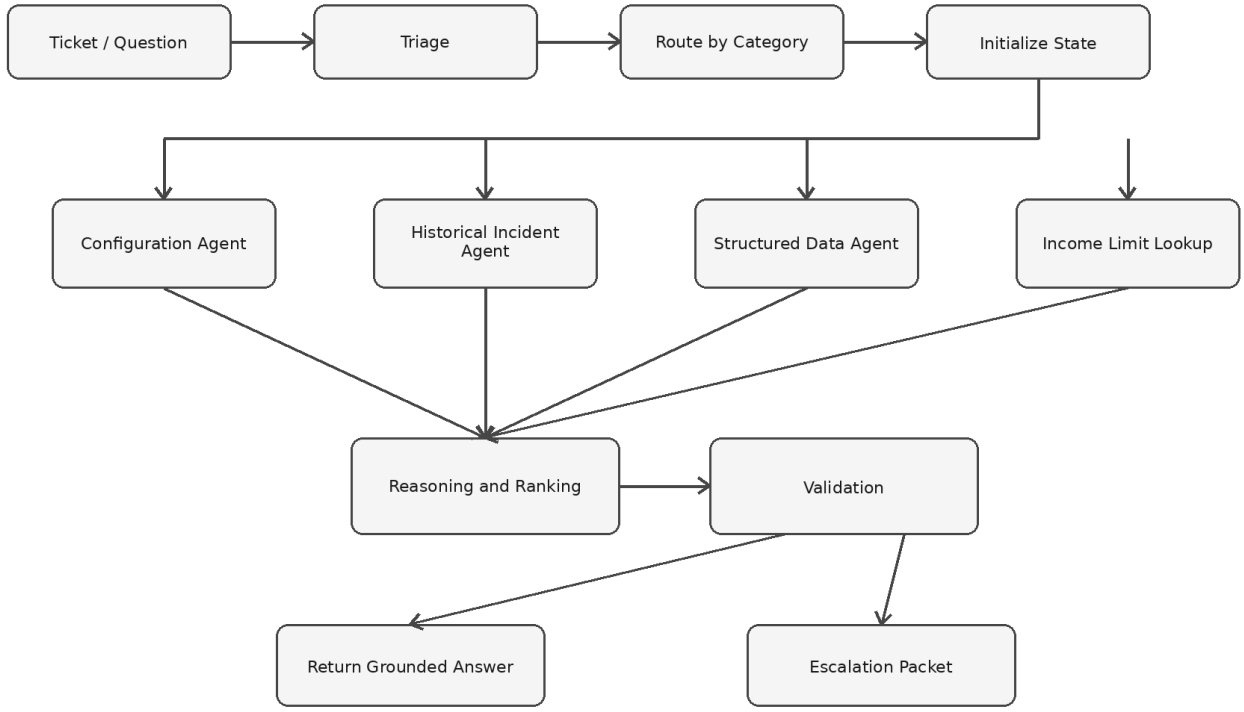
7. FastAPI Application Layer

FastAPI accepts document uploads, support tickets, investigation requests, and tenant self-service questions. Each request is validated at the HTTP boundary, converted into application state, and passed to the LangGraph workflow.

Endpoint Responsibility	Current Behavior	Production Target
Ticket submission	Synchronous workflow execution and complete response	Create investigation record, enqueue work, return investigation ID
Document upload	Accept and process at demo scale	Object storage, malware scanning, async parsing, metadata lifecycle
Tenant question	Invoke constrained tenant workflow	Authenticated tenant context with policy and data-scope enforcement
Investigation result	Returned in the initial request	Status polling or event-driven completion

Synchronous execution is a conscious current limitation. It reduces infrastructure and debugging overhead while the workflow is small. It becomes unsuitable when ingestion, OCR, large retrieval, or multiple external calls can exceed normal HTTP timeouts.

8. LangGraph Orchestration and Shared State



LangGraph maintains a TypedDict-based state throughout the investigation. Nodes are application functions with narrow responsibilities. State is additive: each node contributes evidence or decisions without hiding the original input.

State Category	Representative Fields	Purpose
Request context	ticket_description, module, property	Preserve original issue and routing context
Classification	triage_category, route decisions	Control conditional execution
Evidence	settings_found, similar_tickets, structured_data, income_limit_result	Collect source-specific facts
Reasoning	ranked_causes, validated_causes	Separate hypothesis generation from validation
Control	escalation_flag, escalation_packet	Prevent unsupported finalization

8.1 Why a Graph Instead of a Single Chain

The investigation is conditional. Eligibility issues may require an income-limit lookup; a document question may not. A known production issue may be answered from an incident match; a configuration issue should prioritize settings and release history. LangGraph makes these branches explicit and keeps the state inspectable.

9. Triage and Conditional Routing

BUILT - The current triage node classifies tickets into eligibility, configuration, calculation, document question, or known production issue using keyword matching.

TARGET - Structured entity extraction for property, household, program, jurisdiction, certification date, and error code is a target capability.

Keyword routing is deterministic and easy to debug, but brittle. It can fail on paraphrases, multi-issue tickets, missing context, and domain-specific language. Its value in the current build is architectural: it proves conditional routing and allows the remaining workflow to be exercised.

Category	Typical Trigger	Likely Nodes
Eligibility	Income, qualifies, ineligible, limit	Configuration, incident, structured data, income-limit lookup
Configuration	Setting, flag, property option, override	Configuration and release history
Calculation	Amount, percentage, result mismatch	Structured data and deterministic rule context
Document question	Clause, notice, form, required document	Future document retrieval path
Known production issue	Error pattern or repeated symptom	Historical incident and release knowledge

10. Configuration Agent

The Configuration Agent queries property settings and release notes from SQLite. It is central to the motivating incident because the correct root cause existed outside the Affordable Housing rule path.

- Filter by property and relevant module context
- Return setting name, current value, expected interpretation, and release association
- Preserve evidence identifiers for later validation
- Avoid semantic retrieval where exact setting names and values are available

10.1 Configuration Evidence Strength

A current setting value is direct evidence. It should generally outrank a historical similarity match because it describes the actual environment being investigated. The reasoning layer can still preserve alternative causes when symptoms conflict.

11. Historical Incident Agent

BUILT - Historical tickets are stored in SQLite and retrieved using keyword overlap.

TARGET - Dense embeddings, structured incident metadata, confirmed-resolution tiers, and reranking are target capabilities.

The agent supports organizational learning by retrieving similar symptoms and previous resolutions. The current method tokenizes the new ticket and compares vocabulary with stored tickets. This is transparent but limited: two semantically identical incidents may not match if they use different wording.

Incident Tier	Meaning	Intended Use
Raw ticket	User or support description; may be incomplete	Recall and discovery
Resolved ticket	Ticket with documented resolution	Stronger root-cause support
Confirmed knowledge entry	Staff-validated cause, affected scope, fix, and evidence	High-trust reusable operational knowledge

12. Structured Data and HUD Income-Limit Tool

12.1 Structured Data Agent

The Structured Data Agent reads the ticket and related local records to preserve the issue context. In a production integration, this boundary would retrieve property configuration, household composition, certification dates, calculation inputs, and feature-flag values through approved read-only APIs.

12.2 HUD Income-Limit Lookup

BUILT - A real HUD User API client is implemented with bearer-token authentication and documented endpoint shapes. Parsing is tested; a live network call was not exercised in the sandboxed build environment.

The tool is invoked only for eligibility tickets with sufficient jurisdiction context. The result is written to graph state as structured evidence, not converted into embeddings. This preserves freshness and prevents stale limits from being treated as knowledge-base facts.

Input	Examples	Validation Concern
Location context	ZIP code, jurisdiction, county mapping	Ambiguous or missing jurisdiction
Household context	Household size	Unsupported or invalid size
Program context	AMI percentage, program, effective year	Wrong program or effective date
API result	Income-limit values and source metadata	Authentication, schema drift, empty response

13. Retrieval Architecture

BUILT - TF-IDF cosine similarity is the only current unstructured retrieval method.

TARGET - Hybrid dense retrieval plus BM25 and reranking is the target architecture.

TF-IDF is appropriate for a small demonstrator because it is local, deterministic, fast, and requires no model or vector infrastructure. It is weak when the query and source use different terminology, when exact error identifiers and semantic meaning must both be preserved, or when the corpus grows.

Stage	Current	Target
Candidate generation	TF-IDF cosine similarity	Dense embeddings plus BM25
Filtering	Limited structured filters	Module, program, effective year, property, resolution status
Ranking	Single similarity score	Cross-encoder or model-based reranking
Trust weighting	Implicit	Confirmed knowledge entries above raw tickets
Evaluation	Scenario tests	Recall@k, MRR, citation accuracy, known-incident benchmarks

14. Reasoning and Root-Cause Ranking

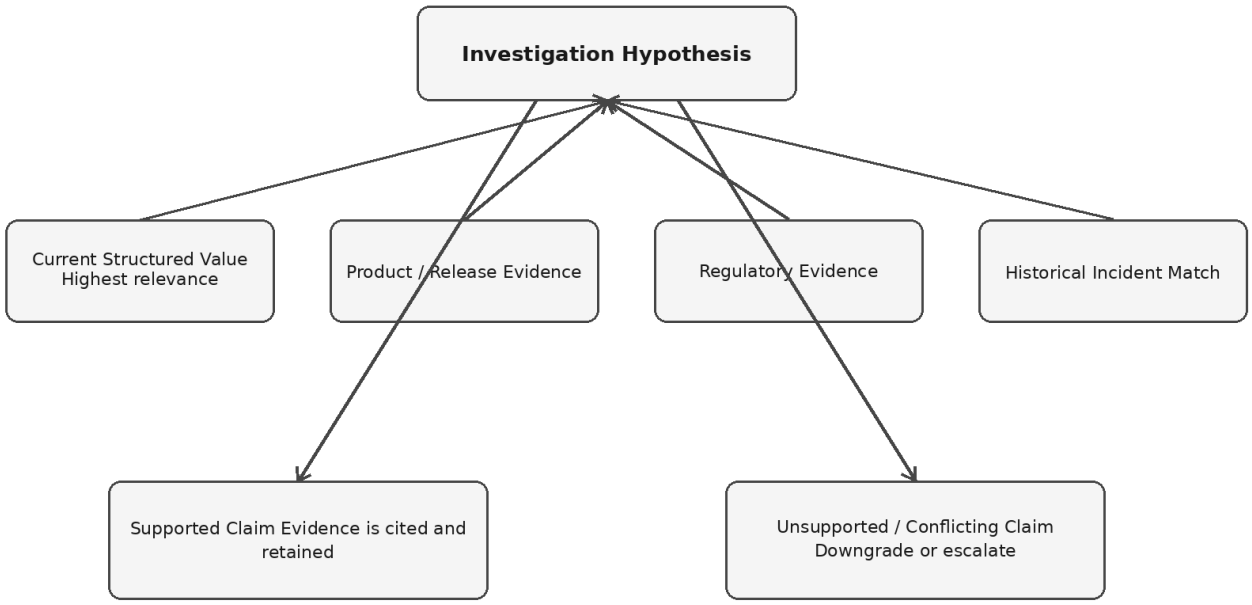
BUILT - The current reasoning node ranks causes from matched settings and similar tickets.

TARGET - Explicit supporting evidence, conflicting evidence, disconfirming checks, and calibrated ranking are target capabilities.

The reasoning layer should not start from general model knowledge. It receives an evidence bundle assembled by the workflow. Its task is to form hypotheses that explain the observed symptom while preserving source references.

Hypothesis Field	Current / Target Behavior
Cause	Short explanation of the suspected root cause
Supporting evidence	Current build retains cited evidence
Conflicting evidence	Target field for facts that argue against the cause
Next check	Target field for a deterministic validation step
Status	Supported, unsupported, conflicting, or requires human review

15. Validation and Evidence Grounding



BUILT - A validation node checks whether each cause has cited evidence. Unsupported claims lose confidence and are flagged rather than presented as facts.

Validation is the most important control separating an investigation assistant from an unconstrained chatbot. The model may propose a plausible cause, but plausibility is not evidence. The workflow requires a setting value, incident, rule, release note, or tool result to support the conclusion.

Evidence Rule

No evidence means no confirmed answer. The result may remain a hypothesis or trigger escalation, but it is not promoted to a factual root cause.

16. Confidence and Escalation

BUILT - Current confidence uses hand-tuned constants such as 0.8 for a matched setting and 0.6 for a similar ticket.

TARGET - Measured confidence based on retrieval relevance, tool success, source agreement, confirmed-incident similarity, and evidence completeness is a target capability.

The current constants make workflow behavior deterministic and testable, but they are not calibrated probabilities. The project explicitly avoids presenting them as statistically measured confidence.

Signal	Why It Matters	Target Treatment
Current structured value	Directly describes the investigated environment	Strong positive evidence

Signal	Why It Matters	Target Treatment
Confirmed historical match	Demonstrates a previously validated failure pattern	Strong when scope and symptoms align
Raw ticket similarity	Useful but may contain speculation	Moderate and trust-weighted
Tool success / freshness	Confirms that current authoritative data was obtained	Required for claims based on live data
Cross-source agreement	Independent evidence supports the same cause	Increase confidence
Contradiction or missing context	Evidence does not converge	Reduce confidence or escalate

16.1 Escalation Packet

When the highest-ranked cause is below the configured threshold, the workflow creates a packet containing the issue summary, evidence reviewed, checks completed, likely causes, and remaining unknowns. The target packet adds explicit rules evaluated and items ruled out so the next engineer does not repeat the same work.

17. Tenant Safety Boundary

BUILT - The Tenant Self-Service Agent is hard-coded never to auto-resolve eligibility, configuration, or calculation questions, regardless of confidence. Explicit safety assertions verify this behavior.

This is not a prompt-only instruction. The application logic enforces the boundary. Tenant-facing outputs can explain what information is needed, provide safe general guidance, or route the issue to qualified staff, but they cannot issue a regulated determination.

Request Type	Permitted Output	Prohibited Output
General document question	Cited explanation or source location	Invented legal conclusion
Eligibility question	Information required and escalation path	Automatic eligibility decision
Configuration issue	Acknowledge issue and route to staff	Expose internal settings or change configuration
Calculation mismatch	Explain that review is required	Declare the financial result correct or incorrect without authoritative validation

18. Data Model and Storage

BUILT - SQLite stores tickets, property settings, release notes, knowledge schema, and investigation-related records.

TARGET - PostgreSQL, pgvector, object storage, Redis, and RabbitMQ are target components, not current infrastructure.

SQLite is a deliberate engineering choice at current scale. It enables reproducible local development, simple tests, and low operational overhead. The schema is relational and designed to make a PostgreSQL migration straightforward when concurrency, tenancy, audit, or corpus size requires it.

Entity	Purpose
tickets	Original support issue, module, property, status, and resolution context
property_settings	Current setting names and values associated with properties
release_notes	Feature or configuration history tied to releases
knowledge_entries	Schema for confirmed, staff-validated root causes and reusable knowledge
investigation state / output	Evidence, ranked causes, validation, and escalation result

19. API and Execution Model

The current graph runs in the same process as the FastAPI request. Python functions query SQLite and call external services. This provides a clear call stack and simple debugging. It also means long-running work blocks the request, retries are application-local, and horizontal scaling is limited by process state.

Concern	Current Model	Target Model
Execution	Synchronous request / response	Queued investigation job
State	In-memory graph plus local persistence	Durable investigation state
Retries	Function-level handling	Worker retry policy and dead-letter handling
Progress	Returned only at completion	Investigation ID and progress events
Scale	Single service instance suitable for demo	Stateless API plus scalable workers

20. Worked Investigation

Input ticket: "The household income is within the limit, but the resident is still showing as ineligible."

1. Triage classifies the ticket as an eligibility issue.
2. The workflow loads the property and ticket context.
3. The Configuration Agent finds the global_verification_required property setting and the release note associated with its introduction.
4. The Historical Incident Agent finds past tickets with similar symptoms.
5. The Income Limit node is eligible to run when jurisdiction context is available.
6. The reasoning node ranks the configuration cause above generic eligibility causes.
7. The validation node retains the cause because it is supported by current setting evidence and historical context.

8. The final result explains that the income rule appears consistent while the property configuration is the leading cause, and it preserves the supporting evidence.

Why the Example Matters

This scenario is implemented, tested, and reproducible in the current build. It demonstrates the central product value without depending on target-only hybrid retrieval or a full regulatory corpus.

21. Testing Strategy

The project tests behavior at the node, workflow, and safety-boundary levels. The goal is not only to verify generated text but to verify state transitions and non-negotiable controls.

Test Layer	Examples
Unit tests	Triage classification, setting lookup, similarity calculation, HUD response parsing
Workflow tests	Eligibility path invokes required nodes and produces ranked, validated causes
Safety assertions	Tenant eligibility, configuration, and calculation requests never auto-resolve
Evidence tests	Unsupported causes are downgraded or removed
Scenario tests	Motivating misconfiguration ticket ranks the correct setting cause highest

21.1 Target Evaluation Suite

- Curated tickets with confirmed root causes
- Retrieval recall and ranking benchmarks
- Citation correctness and unsupported-claim rate
- Tool-call contract tests and schema drift tests
- Escalation quality reviewed by support and engineering staff
- Regression gates before prompt, model, retriever, or graph changes

22. Reliability and Failure Handling

Failure Mode	Current Handling	Required Production Control
HUD API unavailable	Return no live result and preserve uncertainty	Timeouts, retries, circuit breaker, freshness metadata
No setting match	Continue with remaining evidence	Explicit unknown state; avoid default assumptions
No incident match	Do not fabricate history	Fallback retrieval and human escalation
Conflicting evidence	Limited current structure	Preserve contradictions and require disconfirming checks

Failure Mode	Current Handling	Required Production Control
LLM unavailable	Workflow cannot complete reasoning	Deterministic partial report or queued retry
Long-running request	Synchronous timeout risk	Asynchronous worker execution

A production investigation platform should return a partial evidence report when one source fails. Failure of a historical search should not discard current configuration evidence. The graph state model supports this direction because evidence sources are represented independently.

23. Security, Privacy, and Access Control

TARGET - The current demonstrator does not yet implement enterprise authentication, tenant isolation, or production secrets management.

The product domain may include property, customer, household, and certification information. A production deployment therefore requires identity, authorization, data minimization, and full auditability.

- Role-based access to support, engineering, QA, and tenant workflows
- Tenant and property filters enforced before retrieval or tool execution
- Read-only service credentials with least privilege
- Secrets stored outside source code
- PII masking in prompts, logs, and evaluation datasets
- Audit logs for source access, tool execution, and final outputs
- Prompt-injection controls for uploaded or retrieved content
- Tool allowlists and strict input schemas

24. Observability and Evaluation

Category	Metrics
Application	Request latency, failure rate, active investigations, queue depth when asynchronous
Retrieval	Recall@k, ranking quality, empty-result rate, filter selectivity
Tools	Success rate, latency, timeout rate, response freshness, schema failures
Reasoning	Supported-claim rate, contradiction rate, root-cause ranking accuracy
Safety	Tenant escalation rate, prohibited auto-resolution count, access-policy violations
Business	Mean time to resolution, repeat-ticket reduction, engineering escalation quality

The most meaningful product metric is reduction in investigation time without increasing incorrect conclusions. A fast answer that sends support toward the wrong module is negative value. Evaluation must therefore combine speed, correctness, evidence quality, and escalation behavior.

25. Deployment and Scaling

25.1 Current Deployment Shape

- Single FastAPI application
- SQLite database
- Local or application-managed retrieval index
- Synchronous LangGraph execution
- External HUD API call through application-owned client

25.2 Target Deployment Shape

- Stateless API instances behind a load balancer
- PostgreSQL for users, investigations, audit, and structured knowledge
- pgvector or dedicated search service for embeddings
- Object storage for original documents
- Background workers and RabbitMQ for ingestion and investigations
- Redis only where caching or ephemeral coordination is justified
- Central telemetry, evaluation pipeline, and release gates

Scaling should follow measured workload. The architecture does not require microservices merely because it uses agents. The first production boundary should be asynchronous work, because ingestion and multi-source investigation have different latency and retry characteristics from the user-facing API.

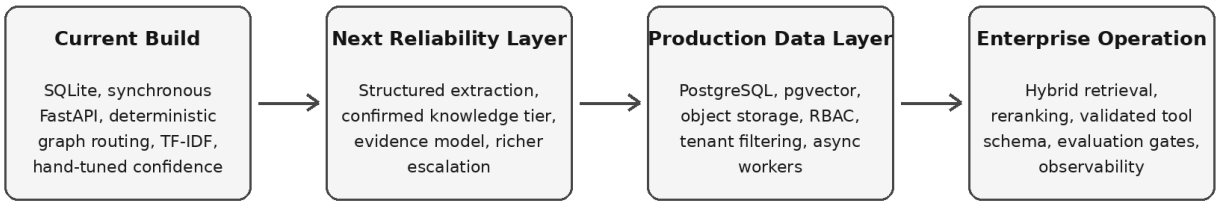
26. Design Decisions and Trade-offs

Decision	Benefit	Accepted Trade-off
SQLite for current build	Low setup cost, deterministic tests, portable demo	Limited concurrency and no enterprise tenancy
Deterministic graph routing	Inspectable, testable, no arbitrary tool access	Less flexible than model-selected tools
TF-IDF retrieval	Simple, local, transparent	Weak semantic recall
Synchronous execution	Simple debugging and deployment	Timeout and scale limitations
Read-only system	Strong safety and trust boundary	Cannot automatically remediate issues
Hand-tuned confidence	Predictable workflow behavior	Not calibrated or evidence-derived
Separate source types	Correct freshness and trust semantics	More orchestration and integration work

27. Current Limitations

- No production authentication, authorization, or tenant-level filtering.
- No authoritative indexed HUD or LIHTC regulatory corpus.
- Historical incident retrieval misses paraphrases because it relies on keyword overlap.
- Triage produces a category rather than a full structured ticket representation.
- Live production settings and certification services are represented by local SQLite reads.
- The HUD client is implemented and parsing-tested, but the sandbox did not exercise a live network request.
- Routing is fixed by graph edges rather than a formal validated model tool-call contract.
- Reasoning stores supporting evidence but not structured conflicting evidence.
- Confidence values are constants, not calibrated measures.
- Execution is synchronous and not suitable for long-running enterprise workloads.

28. Modernization Roadmap



Phase	Primary Deliverables
1. Knowledge quality	Structured entity extraction, confirmed knowledge entries, incident resolution feedback loop
2. Retrieval quality	Dense embeddings, BM25, metadata filters, reranking, retrieval benchmarks
3. Tool governance	Formal tool request schema, allowlists, input validation, timeouts, freshness and provenance
4. Production platform	PostgreSQL, pgvector, object storage, async workers, RBAC, audit trail
5. Evaluation and operations	Root-cause benchmark, citation scoring, observability, deployment gates, human review analytics

29. Lessons Learned

The hardest support problems are often knowledge-routing problems

The rule may be correct while a hidden dependency in another module changes behavior.

Not all data belongs in RAG

Current structured values should be retrieved from authoritative tools, not embedded and allowed to become stale.

Agentic architecture is orchestration, not agent count

The value comes from explicit state, conditional evidence collection, validation, and safe termination.

Safety must exist in code

A prompt instruction is weaker than an enforced application boundary and an automated safety assertion.

Confidence without measurement is only a heuristic

Hand-tuned values are acceptable for deterministic flow control, but they should not be represented as calibrated certainty.

Human escalation is part of reliability

A structured unknown is better than a fluent unsupported answer.

Build infrastructure when the workload requires it

The project gains credibility by using SQLite and synchronous execution honestly rather than claiming unused distributed components.

Appendix A. Capability Status Matrix

Capability	Status	Notes
Four-source knowledge model	Partially built	Architecture exists; regulatory corpus and broad product-doc retrieval remain target
FastAPI service	Built	Synchronous endpoints
LangGraph shared state	Built	TypedDict state across nodes
Keyword triage	Built	Five categories; not structured entity extraction
Configuration Agent	Built	SQLite settings and release notes
Historical Incident Agent	Built, limited	Keyword overlap only
Structured Data Agent	Built	Local records; production integrations target
HUD income-limit client	Built, partially exercised	Parsing tested; live sandbox network call not exercised
TF-IDF retrieval	Built	No dense retrieval, BM25, or reranking
Reasoning and ranking	Built, simplified	Supporting evidence only
Validation node	Built	Unsupported claims downgraded
Confidence scoring	Heuristic	Hand-tuned constants
Escalation packet	Built	Simpler than target packet
Tenant safety boundary	Built and tested	No regulated auto-resolution
PostgreSQL / pgvector / RabbitMQ	Target	Not current infrastructure

Appendix B. Shared Investigation State Reference

Field Group	Examples	Lifecycle
Input	ticket_description, module, property	Created at request boundary
Routing	triage_category, jurisdiction availability	Written by triage / pre-processing
Configuration evidence	settings_found, release context	Written by Configuration Agent
Incident evidence	similar_tickets, match information	Written by Historical Incident Agent
Structured evidence	structured_data, income_limit_result	Written by data and tool nodes
Reasoning	ranked_causes	Written after evidence collection
Validation	validated_causes	Written by evidence validation
Escalation	escalation_flag, escalation_packet	Written when confidence or evidence is insufficient

Appendix C. Glossary

Term	Definition
AMI	Area Median Income, used as a basis for housing income and rent limits.
HUD	United States Department of Housing and Urban Development.
LIHTC	Low-Income Housing Tax Credit program governed primarily under Section 42.
RAG	Retrieval-Augmented Generation: grounding model output in retrieved source content.
LangGraph	A graph-based workflow orchestration library used for stateful, conditional agent workflows.
TF-IDF	A statistical text representation used in the current build for similarity retrieval.
Grounded claim	A conclusion linked to identifiable supporting evidence.
Escalation packet	A structured handoff containing issue context, checks, evidence, hypotheses, and unknowns.

Source note: This case study is based on the documented current LeaseI codebase status and the project origin described by the author. Target architecture is intentionally labeled and is not represented as implemented experience.